



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

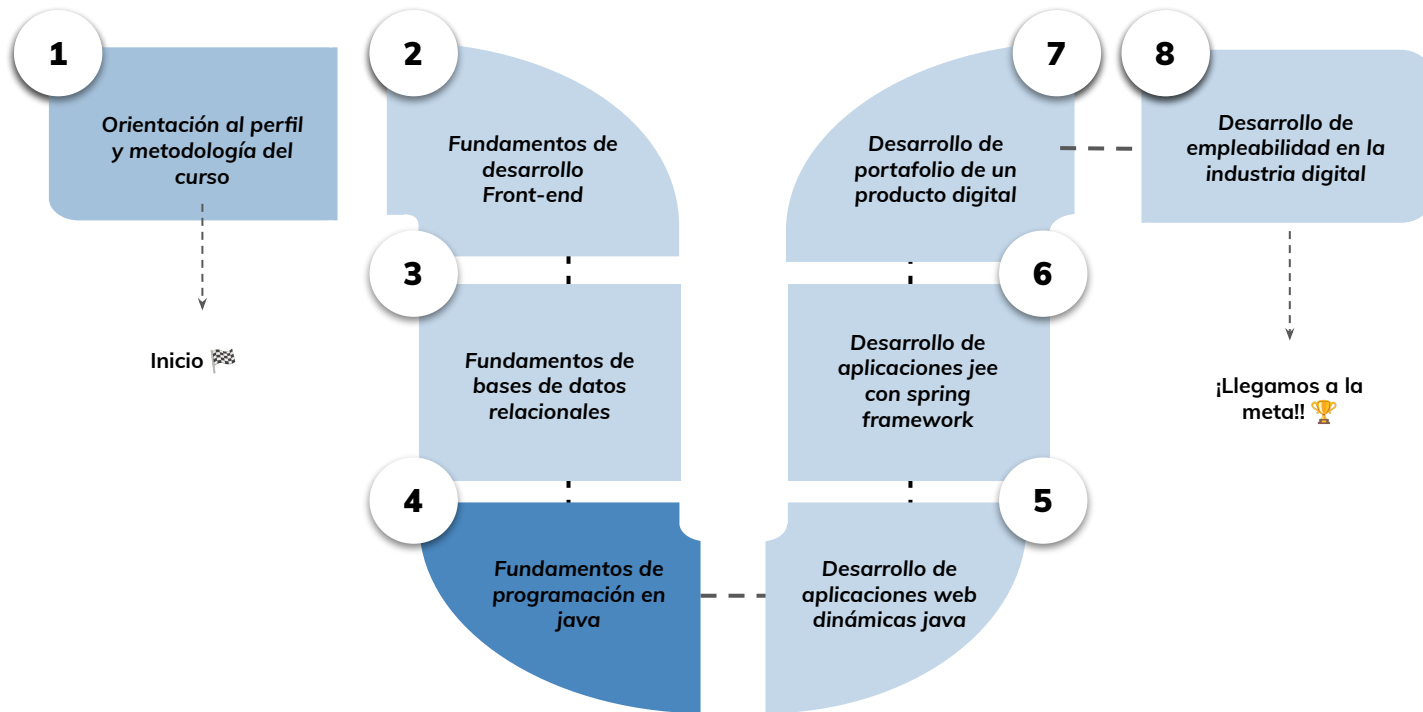


› Algoritmos- Parte 1

Aprendizaje Esperado 2: Estructurar un algoritmo lógico utilizando estructuras de control y expresiones para dar solución a un problema de baja complejidad acorde al lenguaje java.

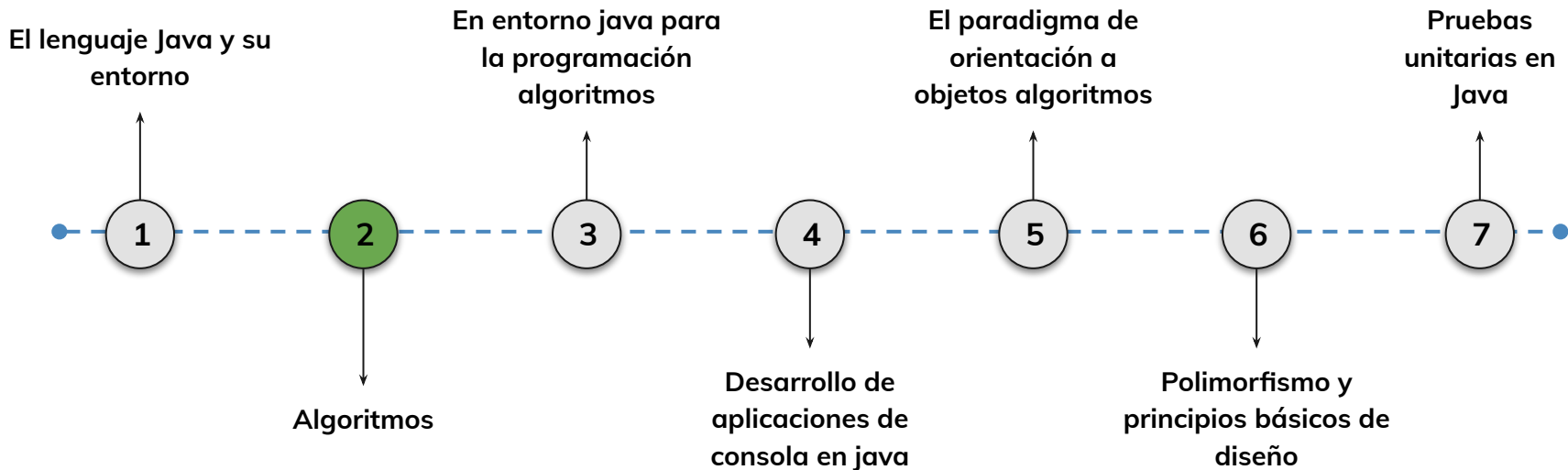
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

1.

Algoritmos: Concepto, componentes y aplicación

En esta clase exploraremos qué es un algoritmo, sus partes fundamentales, cómo se estructuran con variables y tipos de datos, el uso de expresiones aritméticas y estructuras condicionales, y cómo todo esto se traduce en lógica computacional. Se abordarán ejercicios prácticos usando pseudocódigo y lenguaje Java.

Fundamentos de algoritmos

Definición, partes y características

Variables y tipos de datos

Operadores y estructuras condicionales

Expresiones y operadores aritméticos

Condicionales simples, anidados y múltiples



Objetivos de aprendizaje

¿Qué aprenderás?



- Reconocer la estructura básica de un algoritmo.
- Declarar variables y tipos de datos adecuados.
- Aplicar operadores aritméticos para resolver cálculos.
- Utilizar estructuras condicionales para tomar decisiones.
- Implementar lógica computacional en pseudocódigo o Java.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Comprendimos qué es un IDE y su rol en el desarrollo moderno con Java.
- Instalamos y configuramos Eclipse IDE como entorno de desarrollo.
- Conocimos las herramientas principales del ecosistema Java (JDK, JRE, Bytecode, JVM).
- Creamos un proyecto empresarial básico en Java utilizando Eclipse.

➤ Algoritmos: concepto y características

< > ¿Qué es un algoritmo?

**Los algoritmos son una parte fundamental de la programación y la informática.
Pero ¿qué es un algoritmo exactamente?**

Un algoritmo es una secuencia finita de instrucciones bien definidas y no ambiguas, que permite realizar una tarea específica. Es como una receta o manual de pasos para resolver un problema particular. Por ejemplo, las instrucciones para instalar un programa en la computadora, las indicaciones para llegar a un lugar, o los pasos para hacer una tarta, son todos algoritmos.

En informática, los algoritmos nos permiten dar soluciones a problemas de procesamiento de información de manera precisa y automatizada. Son utilizados para el diseño de programas y software.



< > Partes de un algoritmo

Los algoritmos diseñados **son independientes del lenguaje** , es decir, son simplemente instrucciones que se pueden implementar en cualquier lenguaje y, sin embargo, el resultado será el mismo.

Partes de un algoritmo

- **Entrada** : son todos los datos que se introducen al algoritmo para que pueda ejecutarse en base a dicha información.
- **Procesamiento** : con lo que ha recibido en la entrada, el algoritmo llevará a cabo una serie de cálculos para dar con la solución al problema.
- **Salida** : los resultados que se han obtenido del procesamiento de datos se mostrarán en la salida del algoritmo.



< > Características de los algoritmos

Principales características:

- **Secuenciales** : se procesan consecutivamente. Responden a una secuencia de pasos determinados.
- **Precisos** : tienen que ser objetivos en la resolución del problema.
- **Ordenados** : deben leerse y ejecutarse en un orden específico.
- **Finitos** : deben contar con un número concreto de pasos.
- **Concretos** : deben mostrar una solución al problema especificado.
- **Definidos** : antes los mismos datos de entrada, siempre debe conseguirse los mismos datos de salida.



➤ Variables

< > ¿Qué es una variable?

Los datos que vamos a manipular en los algoritmos deben estar contenidos en una variable.

Las variables son **contenedores** que almacenan valores que pueden cambiar. Es un espacio en la memoria de la computadora donde se almacena temporalmente un dato durante la ejecución del programa.

A estas se les asigna un **identificador** o nombre, y el **tipo de dato** que van a almacenar.

El valor de las variables **puede modificarse** durante la ejecución.

Las **constantes** también almacenan valores, pero su contenido no cambia durante la ejecución.



< > Características de las variables

Principales características:

- Deben comenzar con letra.
- No deben contener espacios en blanco.
- No deben contener meta caracteres.
- No se podrán utilizar palabras reservadas como nombres de variable.
- Los identificadores de las variables deben ser únicos e irrepetibles.

Al declarar variables, es recomendable asignar un **valor inicial** :

- Ayuda a evitar errores y resultados inesperados.
- Permite documentar el propósito de la variable según el valor inicial.

Ejemplo: Entero cantidad = 0



Variables locales y globales

Variables Locales:

- Se declaran dentro de un bloque de código como una función o bucle.
- Su ámbito está limitado a ese bloque, no existen fuera de él.
- Se crean y destruyen cada vez que se ejecuta el bloque de código.
- No pueden ser accedidas desde afuera del bloque que las contiene.
- Permiten reutilizar nombres de variables en diferentes partes.

Variables Globales:

- Se declaran fuera de cualquier bloque, al inicio del programa.
- Su ámbito es toda la ejecución del programa, pueden usarse en cualquier parte.
- Se crean al comenzar el programa y se destruyen al finalizar.
- Pueden ser accedidas y modificadas desde cualquier parte del código.
- Deben tener nombres únicos en todo el programa.



➤ Tipos de Datos

< > ¿Qué son los tipos de datos?

Un dato es cualquier elemento que necesitemos procesar en un programa. Para saber qué operaciones podemos realizar con estos datos, los clasificamos según su tipo.

Un tipo de datos es un conjunto de valores que tienen una característica en común y que responden a unas operaciones determinadas.

Dependiendo del lenguaje de programación se puede trabajar con unos tipos u otros. Por ejemplo hay lenguajes que distinguen entre números enteros y números decimales y otros en los que solamente se tiene el tipo de datos numérico, englobando tanto decimales como enteros en el mismo saco.



< > Tipos de

Declaración

En Java esta es la forma de declarar una variable con su respectivo tipo de dato



En todos los lenguajes de programación encontramos una clasificación de tipos de datos siempre presente, **los tipos de datos simples y los tipos de datos compuestos**.

Tipos de Datos

SIMPLES:

También llamados tipos de datos primitivos, son aquellos que contienen un elemento único de un tipo de datos particular y no se pueden descomponer en varios datos independientes.

Ejemplos:

- **Entero** : 5
- **Real** : 5,5
- **Cadena** : "hola mundo"
- **Caracter** : 'H'
- **Booleano** : Verdadero/Falso

COMPUESTOS:

También llamados tipos de datos complejos o estructurados, se componen de agrupaciones de tipos de datos simples.

Ejemplos:

- **Arreglos** : es un conjunto de valores que se almacenan en una misma referencia de la memoria
- **Objetos** : podemos tener distintos tipos de datos simples, y además existen operaciones asociadas a esos objetos.

➤ Expresiones aritméticas

< > Expresiones aritméticas

Las expresiones aritméticas son similares a fórmulas matemáticas.

En el código de la computadora, los operadores y las funciones se ejecutan a través de conjuntos de expresiones que incluyen expresiones aritméticas, de caracteres y booleanas. Estos proporcionan la base para los tipos de trabajo de datos que se realizan dentro de un programa de software.

Existen diferentes tipos de expresiones aritméticas en la sintaxis de programación: enteros o números reales, y números de punto flotante (para identificar y almacenar números complejos que pueden no encajar en un valor entero).



< > Operadores aritméticos

Las operadores aritméticos son:

- + Suma
- - Resta
- * Multiplicación
- / División
- -, **, ^ Exponenciación
- div División entera (cociente)
- mod Módulo (resto de una división)

Los operadores se utilizan de igual forma que en matemáticas. Por consiguiente:

A x B se escribe en algoritmo como **A * B**, y **1/4 x C** como **C/4**.

Al igual que en matemáticas, el signo menos juega un doble papel, como resta en **A - B** y como cambio de signo en **- A**.



< > Operadores aritméticos

Operador DIV: $A \text{ div } B = x$

Obtiene la parte entera de A/B .

Operador MOD: $A \text{ mod } B = x$

Obtiene el resto (módulo) de A/B

<u>División</u>	<u>División entera</u>
$\begin{array}{r} 63.0 \overline{) 03} \\ 10 \\ \underline{0} \end{array}$	$\begin{array}{r} 63 \overline{) 03} \\ 1 \end{array}$
$31.5 \leftarrow \text{Cociente } (/)$	$31 \leftarrow \text{Cociente } (//)$
	$1 \leftarrow \text{Resto } (\%)$

< > Operadores aritméticos

Reglas de prioridad

Las expresiones que tienen dos o más operandos, responden a **reglas matemáticas** que permiten determinar **el orden de las operaciones**, se denominan reglas de prioridad o precedencia.

Las operaciones que están encerradas entre **paréntesis se evalúan primero**. Si existen diferentes paréntesis anidados (interiores unos a otros), las expresiones más internas se evalúan primero.



< > Operadores aritméticos

Reglas de prioridad

Las operaciones aritméticas dentro de una expresión suelen seguir el siguiente orden de prioridad:

- Operador exponencial (, o bien **).
- Operadores *, /, \,
- Operadores div y mod.
- Operadores +, -.

En caso de coincidir varios operadores de igual prioridad en una expresión o subexpresión encerrada entre paréntesis, el orden de prioridad en este caso es de izquierda a derecha.





LIVE CODING

Ejemplo en vivo

Sacando cuentas:

Vamos a crear un algoritmo para realizar todas las operaciones aritméticas vistas hasta ahora. Para esto será necesario:

1. **Entrada:** 2 variables numéricas
2. **Proceso:** operaciones aritméticas
3. **Salida:** 1 variable numérica por cada operación.

Tiempo: 15 minutos

➤ Operadores Relacionales

< > Operadores Relacionales

Los operadores relacionales, también conocidos como operadores de comparación o lógicos, son símbolos que se utilizan en programación **para comparar dos valores** y obtener como **resultado un valor lógico** verdadero o falso.

Estos operadores son muy útiles en programación para comparar valores y expresiones, y para evaluar condiciones que dirigen la lógica de un programa. Se utilizan habitualmente para realizar validaciones.





Los principales operadores relacionales son:

- **Mayor >** : Compara si el valor de la izquierda es mayor que el de la derecha. Ejemplo: $7 > 5$ devuelve true.
- **Menor <** : Comprueba si el valor de la izquierda es menor que el de la derecha. Ejemplo: $3 < 8$ devuelve true.
- **Mayor o igual >=** : Verifica si el valor de la izquierda es mayor o igual al de la derecha. Ejemplo: $5 >= 5$ devuelve true.
- **Menor o igual <=** : Revisa si el valor de la izquierda es menor o igual al de la derecha. Ejemplo: $5 <= 10$ devuelve true.
- **Igual ==** : Compara si dos valores son iguales. Ejemplo: $5 == 5$ devuelve true.
- **Distinto !=** : Compara si dos valores son diferentes. Ejemplo: $5 != 7$ devuelve true.

Operadores relacionales	
>	Mayor.
<	Menor.
>=	Mayor o igual.
<=	Menor o igual.
==	Igual.
!=	Distinto.
=	Asignación.



< > Operadores Relacionales

Operador de asignación (=)

El operador (=) es un operador sobrecargado, es decir, que dependiendo del contexto realiza distintas operaciones.

En el contexto de la **asignación** , el operador = se utiliza para modificar el valor de las variables, es decir, asignarle un nuevo valor. Ejemplo: `x = 5`.

Si lo que queremos es utilizar este operador como sinónimo de **igualdad** y no de asignación, se utiliza el operador dos veces: (==)



LIVE CODING

Ejemplo en vivo

Practicando asignaciones:

Declararemos cuatro variables de **tipo entero A, B, C y D** y le asignaremos un valor diferente a cada una. Vamos a utilizar sólo una variable auxiliar.

A continuación, realizar las instrucciones necesarias para que:

- **B** tome el valor de **C**
- **C** tome el valor de **A**
- **A** tome el valor de **D**
- **D** tome el valor de **B**.

Tiempo: 20 minutos



» Teoría de conjuntos y lógica proposicional

< > Teoría de Conjuntos

¿Qué es y para qué sirve?

La teoría de conjuntos es una rama de las matemáticas que estudia las propiedades de los conjuntos y sus operaciones. En programación, la teoría de conjuntos se utiliza para:

- **Modelar datos** : Los conjuntos permiten representar colecciones de valores o datos de manera abstracta. Por ejemplo, el conjunto de todos los usuarios registrados en un sistema.
- **Trabajar con relaciones entre datos** : sirven para relacionar datos de forma lógica y extraer información útil.
- **Resolver problemas algorítmicos** : Muchos problemas computacionales se modelan utilizando conjuntos y sus operaciones, como por ejemplo algoritmos de búsqueda y ordenamiento.



< > Lógica Proposicional

¿Qué es y para qué sirve?

La lógica proposicional se refiere al estudio de las proposiciones y cómo se combinan para formar declaraciones más complejas.

En programación, utilizamos la lógica proposicional para **tomar decisiones basadas en condiciones** . Podemos combinar proposiciones utilizando **operadores lógicos** como **AND**, **OR y NOT** , para evaluar si una afirmación es **verdadera** o **falsa** , y así controlar el flujo de nuestro programa.



< > Expresiones lógicas

Son declaraciones que evalúan si una afirmación es verdadera o falsa. Utilizamos operadores lógicos como AND, OR y NOT para combinar y comparar valores.

- **Conjunción** : Si queremos verificar si dos condiciones se cumplen simultáneamente, usamos el operador **AND**. El simbolismo de referencia es **&&**.
- **Disyunción** : Si queremos verificar si al menos una de las condiciones es verdadera, usamos el operador **OR**. El simbolismo de referencia es **||**.
- **Negación** : El operador **NOT** invierte el valor de una expresión. El simbolismo de referencia es **!**.



➤ Tablas de la Verdad

< > Tablas de la Verdad

¿Qué son y para qué se utilizan?:

Consisten en una tabla que relaciona todas las combinaciones posibles de valores de verdad de las variables o proposiciones de una expresión, con el resultado de evaluar la expresión completa. En programación se utilizan para diseñar algoritmos y probar que ciertas condiciones se cumplen según lo esperado.

Las tablas de verdad sirven para:

- Comprobar si un argumento es válido.
- Demostrar equivalencias entre expresiones.
- Verificar circuitos lógicos y digitales.
- Simplificar expresiones booleanas.
- Modelar problemas de lógica.



< > Tablas de la Verdad

Conjunción &&, AND:

La conjunción de dos proposiciones es **verdadera únicamente** en el caso en que **ambas proposiciones lo sean**.

Columnas representadas:

- **p** : Premisa 1
- **q** : Premisa 2
- **p && q**: Conclusión o resultado

p	q	p && q
V	V	V
V	F	F
F	V	F
F	F	F



< > Tablas de la Verdad

Disyunción $\parallel$, OR:

La disyunción entre dos proposiciones es **verdadera** cuando **al menos una de dichas proposiciones lo es** .

Columnas representadas:

- **p** : Premisa 1
- **q** : Premisa 2
- **p $\parallel$ q**: Conclusión o resultado

p	q	p $\parallel$ q
V	V	V
V	F	V
F	V	V
F	F	F



< > Tablas de la Verdad

Negación !, NOT:

El valor de verdad de la **negación** de una proposición **verdadera es falso** y el valor de verdad de la negación de una proposición **falsa es verdadero** .

Columnas representadas:

- **p** : Premisa 1
- **!p** : Negación premisa 1

p	!p
V	F
F	V



LIVE CODING

Ejemplo en vivo

Implementando Tablas de la Verdad:

Supongamos que estamos desarrollando un sistema que recomienda películas según las preferencias del usuario.

Las reglas son:

- Si al usuario le gustan las películas de acción **O** las películas de ciencia ficción, recomendar Alien.
- Si al usuario le gustan las películas románticas **Y** las películas de drama, recomendar Titanic.

Tiempo: 25 minutos

LIVE CODING

Ejemplo en vivo

Construimos tablas de verdad para evaluar en qué casos se cumplen R1 y R2:

Podemos representar estas reglas como:

- A: Al usuario le gustan películas de acción
- B: Al usuario le gustan películas de ciencia ficción
- C: Al usuario le gustan películas románticas
- D: Al usuario le gustan películas de drama
- **R1:** Recomendar Alien = **A OR B**
- **R2:** Recomendar Titanic = **C AND D**



LIVE CODING

Ejemplo en vivo

Aquí podemos ver los posibles valores para R1 y R2:

A	B	R1 (A OR B)
V	V	V
V	F	V
F	V	V
F	F	F

C	D	R2 (C AND D)
V	V	V
V	F	F
F	V	F
F	F	F

➤ Estructura Condicional Anidada

< > Condicionales Anidados

¿Para qué se utilizan?:

Los condicionales anidados permiten **evaluar múltiples condiciones** de forma encadenada y ejecutar diferentes códigos según combinaciones de verdadero y falso. Proporcionan complejidad y versatilidad a la lógica de decisión en los programas.

Se pueden anidar tantos condicionales como se necesite para modelar una situación determinada. El orden de evaluación es **secuencial** y de adentro hacia afuera.

Las condiciones internas sólo se evalúan si todas las condiciones externas fueron verdaderas.



< > Condicional Anidado

“Si - Anidados”:

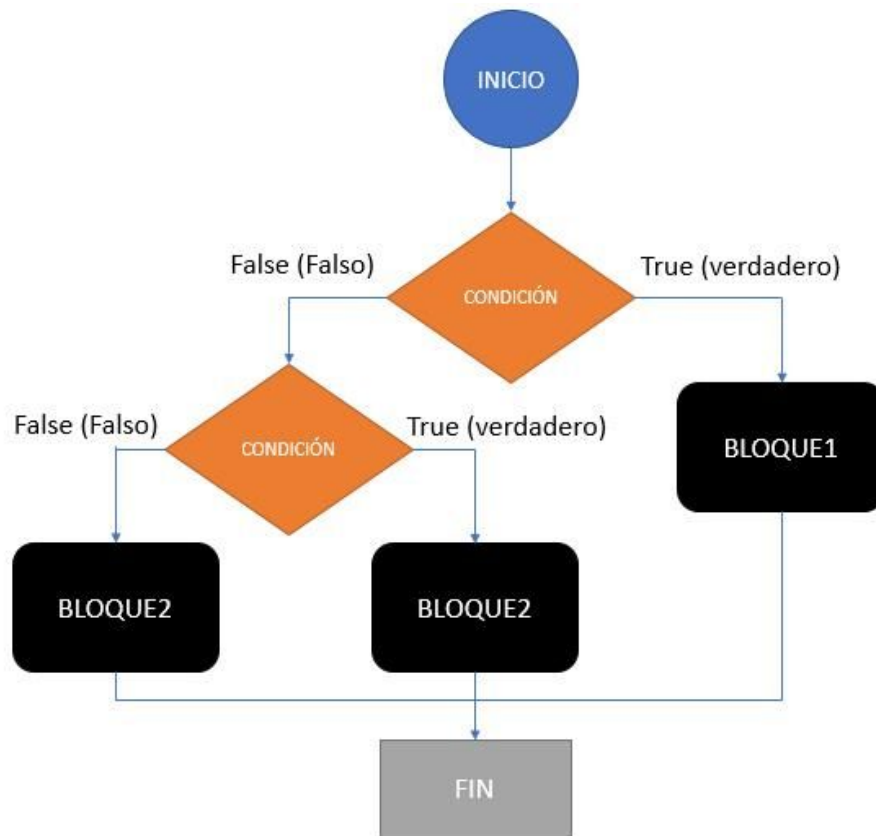
Consisten en tener **una instrucción condicional dentro de otra** .

SI <condición 1> ENTONCES
 Acción 1

SI NO
 SI <condición 2> ENTONCES
 Acción 2

SI NO
 Acción 3

FIN SI
FIN SI



LIVE CODING

Ejemplo en vivo

Calculando entradas:

Vamos a crear un algoritmo para una empresa que tiene salas de juegos para todas las edades y quiere calcular de forma automática el precio que debe cobrar a sus clientes por entrar. **El programa debe preguntar al usuario la edad del cliente y mostrar el precio de la entrada.**

1. Si el cliente es menor de 4 años puede entrar gratis
2. Si el cliente tiene entre 4 y 18 años debe pagar 5€
3. Si el cliente es mayor de 18 años, 10€.

Tiempo: 20 minutos

➤ Estructura Condicional Múltiple

< > Condicional Múltiple

Permite **evaluar una expresión** y ejecutar diferentes bloques de código según el valor que coincida.

Las estructuras de selección múltiple son útiles cuando queremos tomar diferentes acciones **dependiendo del valor de una variable** . Es como un menú de opciones donde elegimos una **según lo que necesitamos** . Por ejemplo: "Si el día de la semana es 1, entonces es lunes; si es 2, entonces es martes, y así sucesivamente".

Los condicionales múltiples permiten evaluar múltiples casos de forma directa y organizada, en lugar de usar muchos si - entonces anidados.



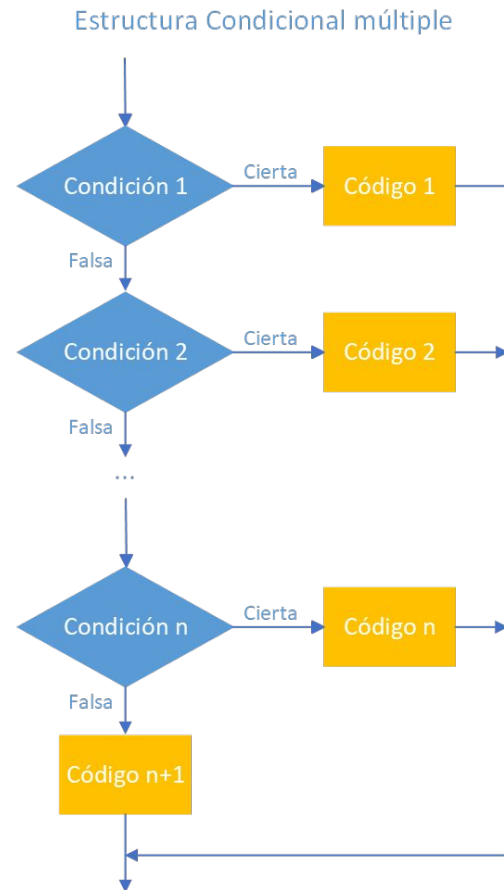
< > Condicional Múltiple

Según:

```
SEGÚN (variable/expresión){  
  caso 1 : println("lunes")  
  caso 2 : println("martes")  
  caso 3 : println("miércoles")  
  caso 4 : println("jueves")  
  caso 5 : println("viernes")  
  caso 6 : println("sábado")  
  caso 7 : println("domingo")  
}
```

La palabra **Según** está antes de las acciones que se realizan si se cumple la condición de la variable expresión. Y, dependiendo del valor de la expresión, define cuál es el “**caso**” que se ejecuta.

Cada caso tiene un bloque de código determinado.



LIVE CODING

Ejemplo en vivo

Frutería:

Vamos a escribir un algoritmo que nos devuelva el valor de cada kg de fruta según la fruta seleccionada. Las opciones disponibles son: “plátano”, “naranja”, “papaya”, “frutilla”, “manzana”.

El usuario debe ingresar el dato como cadena (el nombre de la fruta), y el programa debe imprimir el valor del kg de fruta seleccionada.

Tiempo: 15 minutos



Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

Superliga

Superliga

Contexto: 🙌

Las operaciones aritméticas no solo se aplican en problemas matemáticos, también en escenarios cotidianos como los deportes. Hoy trabajaremos con un algoritmo que simula el cálculo de puntos en una liga de fútbol según los resultados obtenidos por un equipo.

Consigna: 📝

Elabora un algoritmo que permita ingresar el número de partidos ganados, empatados y perdidos por tu equipo favorito. Según estos datos, calcula y muestra el puntaje total:

- Victoria = 3 puntos
- Empate = 1 punto
- Derrota = 0 puntos

Tiempo 🕒: 25 Minutos

Paso a paso: ⚙️

1. Declarar variables: pg, pe, pp, ppg, ppe, ptotal.
2. Leer cantidad de partidos ganados, empatados y perdidos.
3. Calcular $ppg = pg * 3$, $ppe = pe * 1$.
4. Calcular $ptotal = ppg + ppe$.
5. Imprimir el puntaje total del equipo.

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ **Comprendimos qué es un algoritmo y sus partes esenciales.**
- ✓ **Reconocimos tipos de datos y cómo declarar variables.**
- ✓ **Aplicamos expresiones aritméticas con operadores básicos.**
- ✓ **Utilizamos estructuras condicionales simples y anidadas.**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son claves para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

